

Quiz — 2.1.2 Thinking Ahead

Name: _____ Date: _ **Mark:** ____ / 20

OCR H446 · Component 02 · 2.1.2

Section A — Multiple choice (6 marks)

1. A program performs a binary search on a list of customer IDs. The precondition that must hold before the search runs is that the list is: A. Empty B. Sorted into order C. Stored in a database D. Displayed on screen Your answer: ☐
 2. In an online shop's "place order" routine, which of the following is an **input**? A. The confirmation email sent to the customer B. The customer's chosen quantity and delivery address C. Validating the card number D. The updated stock total saved to the database Your answer: ☐
 3. A weather website recomputes the same regional forecast for thousands of visitors each minute. Storing the computed forecast for quick reuse is an example of: A. A precondition B. Decomposition C. Caching D. A race condition Your answer: ☐
 4. Which is the main trade-off a developer must accept when adding caching to the weather site? A. Cached data may become stale and serve out-of-date forecasts B. The site can never use the cache again C. Caching always makes the site slower D. Caching removes the need for inputs Your answer: ☐
 5. A studio reuses a pre-written, pre-tested physics module in three different games. A key benefit of using such a reusable component is: A. It guarantees the games run concurrently B. Faster development with fewer bugs because the module is already tested C. It removes the need for any inputs or outputs D. It makes the games' data automatically sorted Your answer: ☐
 6. Which of these is an **output** of a cash machine's "withdraw cash" routine, rather than an input or a process? A. The PIN typed by the user B. The amount requested by the user C. The dispensed banknotes and updated balance D. Checking the balance is sufficient Your answer: ☐
-

Section B — Short answer (9 marks)

7. A login routine must run before a user can view their account page. State the **precondition** for displaying the account page, and explain what could go wrong if the routine ran without checking it. [2]

8. A flight-booking website caches its currency-conversion rates. Discuss **one** benefit and **one** trade-off of caching the rates in this scenario. [2]

9. For a "calculate total price" routine in a shopping basket, identify **two** inputs and **one** output. Explain why "apply the discount" is *not* counted as an input or output. [3]

10. A company is building three apps that all need a date-validation feature. Explain **two** benefits of writing this as a single reusable component rather than rewriting it each time. **[2]**

Section C — Applied question (5 marks)

12. A developer is planning a routine that displays a personalised news feed when a user opens a social-media app. Before writing any code, describe how *thinking ahead* would shape the design. Refer to: the routine's **inputs and outputs**, a **precondition** that must hold first, what data could usefully be **cached**, and one **reusable component** that could be used. **[5]**

Answer key

Section A (6 marks — 1 each) 1. **B** — binary search only works correctly on a sorted list; that is its precondition. 2. **B** — quantity and address are data *in*; A and D are outputs; C is a process. 3. **C** — storing an expensive, frequently used result for reuse is caching. 4. **A** — the cached forecast can fall out of date (stale) relative to the live source; that is the classic caching trade-off. 5. **B** — pre-tested modules speed development and carry fewer bugs. 6. **C** — dispensed cash and updated balance are results *out*; A and B are inputs, D is a process.

Section B (9 marks)

7. **[2]** Precondition: the user must be successfully authenticated/logged in (their credentials verified) before the account page is shown (1). If unchecked, an unauthenticated user could view private account data — a security/privacy breach, or the page could error on missing user data (1).
8. **[2]** Benefit (1): rates are read from fast local storage instead of re-fetching from the external service each time, so pages load faster and reduce traffic/cost. Trade-off (1): the cached rate can go stale if the market moves, so customers may be quoted an out-of-date price unless the cache is invalidated/refreshed. Must be applied to the rates scenario.
9. **[3]** Two inputs (1): item prices/quantities; any discount code or voucher entered; VAT rate. Output (1): the final total price (and any change/saving shown). Why "apply the discount" is excluded (1): it is a *process/step* that transforms data, not data flowing in or a result flowing out.
10. **[2]** Any two (1 each): faster development (write once, drop into all three apps); fewer bugs because the component is already tested; consistency — all apps validate dates the same way; easier maintenance — fix once and all apps benefit.

Section C (5 marks) 12. **[5]** One mark each, up to 5: Inputs — user ID, follow/friend list, post timestamps; Output — an ordered list of posts rendered to the feed (1). Precondition — the user must be logged in / their profile loaded before a personalised feed can be built (1). Caching — recently fetched posts or computed feed can be cached so reopening the app is fast and reduces server requests (1), accepting

that the cache may go stale and need refreshing (1 — credit the trade-off if given). Reusable component — e.g. an existing date/time-formatting or image-loading or sorting module reused rather than rewritten, saving time and bugs (1). Must be applied to the feed scenario; generic definitions score poorly.

Total: 6 + 9 + 5 = 20